

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Лукис Вадим Вадимович

Факультет прикладной информатики

Группа К3242

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватель Говорова Марина Михайловна

Цель работы: Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Практическое задание

1. Установите MongoDB для обеих типов систем (32/64 бита).
2. Проверьте работоспособность системы запуском клиента mongo.
3. Выполните методы:
 - a) db.help()
 - b) db.help
 - c) db.stats()
4. Создайте БД learn.
5. Получите список доступных БД.
6. Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.
7. Просмотрите список текущих коллекций.
8. Переименуйте коллекцию unicorns.
9. Просмотрите статистику коллекции.
10. Удалите коллекцию.
11. Удалите БД learn.

Практические задания из ЛР 6.2 (их кратко не перечислить)

Выполнение

ЛР_6.1

Установка MongoDB:

```
C:\Windows\System32\cmd.exe

Microsoft Windows [Version 10.0.19045.6456]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Program Files\MongoDB\Server\8.0\bin>mongod --version
db version v8.0.23
Build Info: {
  "version": "8.0.23",
  "gitVersion": "ccf0d81588377542f00eeecf1b1ffbc095b1eeff",
  "modules": [],
  "allocator": "tcmalloc-gperf",
  "environment": {
    "distmod": "windows"
  }
}

C:\Program Files\MongoDB\Server\8.0\bin>mongosh --version
2.8.3

C:\Program Files\MongoDB\Server\8.0\bin>
```

Запускаем сервер командой: `mongod --dbpath "C:\data\db"`

Подключаемся к серверу:

```
C:\Program Files\MongoDB\Server\8.0\bin>mongosh
Current Mongosh Log ID: 6a203ba58dc7e77d6dabc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.0.23
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-06-03T17:06:20.243+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> _
```

Выполнение методов

```
test> db.version()
8.0.23
test> db.help()

Database Class:

getMongo           Returns the current database connection
getName            Returns the name of the DB
getCollectionNames Returns an array containing the names of
getCollectionInfos Returns an array of documents with collec
runCommand          Runs an arbitrary command on the database
adminCommand        Runs an arbitrary command against the adm
aggregate           Runs a specified admin/diagnostic pipelin
getSiblingDB        Returns another database without modifyin
getCollection        Returns a collection or a view object tha
dropDatabase         Removes the current database, deleting th
createUser           Creates a new user for the database on wh
updateUser           Updates the user's profile on the databas
changeUserPassword   Updates a user's password. Run the method
logout              Ends the current authentication session.
dropUser             Removes the user from the current databas
dropAllUsers         Removes all users from the current databa
auth                Allows a user to authenticate to the data
grantRolesToUser     Grants additional roles to a user.
revokeRolesFromUser  Removes a one or more roles from a user o
getUser              Returns user information for a specified
getUsers             Returns information for all the users in
createCollection     Create new collection
```

```
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
test> _
```

Создание БД

```
test> use learn
switched to db learn
```

Получаем доступные БД

```
learn> show dbs
admin    40.00 KiB
config   92.00 KiB
local    40.00 KiB
learn> _
```

Тут нету learn, потому что мы в нее не вставляли документ

Создаем коллекцию

```
learn> db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a203d8c8dc7e77d6dabc114') }
}
learn> _
```

Смотрим список текущих коллекций

```
learn> show collections
unicorns
learn> _
```

Переименовываем коллекцию

```
learn> db.unicorns.renameCollection("unicorns_renamed")
{ ok: 1 }
learn> _
```

Смотрим статистику коллекции

Там много текста, все не буду показывать

```
learn> db.unicorns_renamed.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVers
0,encryption=(keyid=,name=),exclusive=false,extractor=,format=btree,huffman_key=,huffman_y
al_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_ke
,merge_custom=(prefix=,start_generation=0,suffix=),merge_max=15,merge_min=0),memory_page_t
t=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0),type=fi
type: 'file',
uri: 'statistics:table:collection-7-7231830086981458783',
LSM: {
  'bloom filter false positives': 0,
  'bloom filter hits': 0,
  'bloom filter misses': 0,
  'bloom filter pages evicted from cache': 0,
  'bloom filter pages read into cache': 0,
  'bloom filters in the LSM tree': 0,
  'chunks in the LSM tree': 0,
  'highest merge generation in the LSM tree': 0,
  'queries that could have benefited from a Bloom filter that did not exist': 0,
  'sleep for LSM checkpoint throttle': 0,
  'sleep for LSM merge throttle': 0,
  'total size of bloom filters': 0
},
}
```

Удаляем коллекцию

```
}  
learn> db.unicorns_renamed.drop()  
true  
learn>
```

Удаляем БД

```
learn> use learn  
{ ok: 1, dropped: 'learn' }
```

ЛР_6.2

2.1.1

Заполняем коллекцию единорогов

```
test> use learn
switched to db learn
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040b26179384a32abc114') }
}
learn> db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040b86179384a32abc115') }
}
learn> db.unicorns.insert({name: 'Unicrom', loves: ['enegon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040be6179384a32abc116') }
}
learn> db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040c36179384a32abc117') }
}
learn> db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040c96179384a32abc118') }
}
learn> db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040d56179384a32abc11a') }
}
learn> db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040d56179384a32abc11a') }
}
learn> db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040d96179384a32abc11b') }
}
learn> db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040dd6179384a32abc11c') }
}
learn> db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040e16179384a32abc11d') }
}
learn> db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2040e66179384a32abc11e') }
}
learn> _
```

Вставляем документ

```
learn> var doc = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165};

learn> db.unicorns.insert(doc);
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a2041946179384a32abc11f') }
}
learn> db.unicorns.find().pretty()
```

```
learn> db.unicorns.find().pretty()
[
  {
    _id: ObjectId('6a2040b26179384a32abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a2040b86179384a32abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a2040be6179384a32abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a2040c36179384a32abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a2040c96179384a32abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a2040d16179384a32abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a2040d56179384a32abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a2040d96179384a32abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
]
```

```

    },
    {
      _id: ObjectId('6a2040dd6179384a32abc11c'),
      name: 'Leia',
      loves: [ 'apple', 'watermelon' ],
      weight: 601,
      gender: 'f',
      vampires: 33
    },
    {
      _id: ObjectId('6a2040e16179384a32abc11d'),
      name: 'Pilot',
      loves: [ 'apple', 'watermelon' ],
      weight: 650,
      gender: 'm',
      vampires: 54
    },
    {
      _id: ObjectId('6a2040e66179384a32abc11e'),
      name: 'Nimue',
      loves: [ 'grape', 'carrot' ],
      weight: 540,
      gender: 'f'
    },
    {
      _id: ObjectId('6a2041946179384a32abc11f'),
      name: 'Dunx',
      loves: [ 'grape', 'watermelon' ],
      weight: 704,
      gender: 'm',
      vampires: 165
    }
  ]
}
learn>

```

2.2.1

Запросы для вывода самцов/самок

Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```

learn> db.unicorns.find({gender:'f'}).sort({name:1}).limit(3)
[
  {
    _id: ObjectId('6a2040b86179384a32abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a2040d16179384a32abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a2040dd6179384a32abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn>

```

```
learn> db.unicorns.find({gender:'m'}).sort({name:1})
[
  {
    _id: ObjectId('6a2041946179384a32abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a2040b26179384a32abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a2040d56179384a32abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a2040e16179384a32abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a2040d96179384a32abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a2040c36179384a32abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a2040be6179384a32abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn> _
```

Поиск всех самок которые любят carrot

Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```
learn> db.unicorns.findOne({gender:'f', loves:'carrot'})
{
  _id: ObjectId('6a2040b86179384a32abc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({gender:'f', loves:'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a2040b86179384a32abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>
```

2.2.2

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
]
learn> db.unicorns.find({gender:'m'}, {loves:0, gender:0, _id:0})
[
  { name: 'Horny', weight: 600, vampires: 63 },
  { name: 'Unicrom', weight: 984, vampires: 182 },
  { name: 'Rooooooodles', weight: 575, vampires: 99 },
  { name: 'Kenny', weight: 690, vampires: 39 },
  { name: 'Raleigh', weight: 421, vampires: 2 },
  { name: 'Pilot', weight: 650, vampires: 54 },
  { name: 'Dunx', weight: 704, vampires: 165 }
]
learn>
```

2.2.3

Вывести список единорогов в обратном порядке добавления.

```

learn> db.unicorns.find().sort({$natural:-1})
[
  {
    _id: ObjectId('6a2041946179384a32abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a2040e66179384a32abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a2040e16179384a32abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a2040dd6179384a32abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a2040d96179384a32abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a2040d56179384a32abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a2040d16179384a32abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a2040c96179384a32abc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {

```

```

    },
    {
      _id: ObjectId('6a2040c36179384a32abc117'),
      name: 'Rooooooodles',
      loves: [ 'apple' ],
      weight: 575,
      gender: 'm',
      vampires: 99
    },
    {
      _id: ObjectId('6a2040be6179384a32abc116'),
      name: 'Unicrom',
      loves: [ 'energon', 'redbull' ],
      weight: 984,
      gender: 'm',
      vampires: 182
    },
    {
      _id: ObjectId('6a2040b86179384a32abc115'),
      name: 'Aurora',
      loves: [ 'carrot', 'grape' ],
      weight: 450,
      gender: 'f',
      vampires: 43
    },
    {
      _id: ObjectId('6a2040b26179384a32abc114'),
      name: 'Horny',
      loves: [ 'carrot', 'papaya' ],
      weight: 600,
      gender: 'm',
      vampires: 63
    }
  ]
}
learn>

```

2.1.4

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```

learn> db.unicorns.find({gender:'m'}, {name:1, loves:{$slice:1}, _id:0}).sort({name:1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>

```

2.3.1

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender:'f', weight:{$gte:500, $lte:700}}, {_id:0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> 
```

2.3.2

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender:'m', weight:{$gte:500}, loves:{$all:['grape','lemon']}}, {_id:0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> 
```

2.3.3

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({vampires:{$exists:false}})
[
  {
    _id: ObjectId('6a2040e66179384a32abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> 
```

2.3.4

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({gender:'m'}, {name:1, loves:{$slice:1}, _id:0}).sort({name:1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn> _
```

3.1.1

Создаем коллекцию towns

```
learn> db.towns.insert({name:"Punxsutawney", populatiuon: 6200, last_sensus: ISODate("2008-01-31"), famous_for:[""], mayor:{name:"Jim Wehrle"}});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a204c556179384a32abc120') }
}
learn> db.towns.insert({name:"New York", populatiuon: 22200000, last_sensus: ISODate("2009-07-31"), famous_for:["status of liberty","food"], mayor:{name:"Michael Bloomberg", party:"I"}});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a204ca56179384a32abc121') }
}
learn> db.towns.insert({name:"Portland", populatiuon: 528000, last_sensus: ISODate("2009-07-20"), famous_for:["beer","food"], mayor:{name:"Sam Adams", party:"D"}});
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a204cbd6179384a32abc122') }
}
learn>
```

Города с независимыми мэрами

Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({"mayor.party":"I"}, {name:1, mayor:1, _id:0});
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn>
```

Беспартийные мэры

Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({"mayor.party":{"$exists:false"}}, {name:1, mayor:1, _id:0});
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> _
```

3.1.2

Сформировать функцию для вывода списка самцов единорогов.

```
[ { name: 'Dunx', gender: 'm', weight: 500, loves: 'apple' },  
  { name: 'Horny', gender: 'm', weight: 600, loves: 'chocolate' } ]  
learn> var maleQuery = {gender: 'm'};  
  
learn>
```

Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
learn> var cursor = db.unicorns.find(maleQuery).sort({name: 1}).limit(2);  
  
learn>
```

Вывести результат, используя forEach.

```
learn> cursor.forEach(function(doc) {print(doc.name);});  
Dunx  
Horny  
  
learn> _
```

3.2.1

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count();  
2  
learn>
```

3.2.2

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves");  
[  
  'apple',  
  'chocolate',  
  'grape',  
  'papaya',  
  'strawberry',  
  'watermelon',  
  'carrot',  
  'energon',  
  'lemon',  
  'redbull',  
  'sugar',  
]  
learn>
```

3.2.3

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]];
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn>
```

3.3.1

Выполнить команду:

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

```
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'});
TypeError: db.unicorns.save is not a function
learn>
```

Метод `save()` был удалён в MongoDB 8.0, поэтому я использовал `insertOne()` для вставки нового документа.

```
learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a2051b76179384a32abc123')
}
learn> _
```

Проверить содержимое коллекции `unicorns`.

```
learn> db.unicorns.find({name: 'Barney'});
[
  {
    _id: ObjectId('6a2051b76179384a32abc123'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
learn>
```

3.3.2

Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.

```
learn> db.unicorns.update({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}});
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> _
```

Проверить содержимое коллекции *unicorns*.

```
learn> db.unicorns.find({name:'Ayna'});
[
  {
    _id: ObjectId('6a2040d16179384a32abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>
```

3.3.3

Для самца единорога *Raleigh* внести изменения в БД: теперь он любит рэдбул.

```
learn> db.unicorns.update({name:'Raleigh'}, {$push:{loves:'redbull'}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> _
```

Проверить содержимое коллекции *unicorns*.

```
learn> db.unicorns.find({name:'Raleigh'});
[
  {
    _id: ObjectId('6a2040d96179384a32abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> _
```

3.3.4

Всем самцам единорогов увеличить количество убитых вампиров на 5.

```
learn> db.unicorns.update({gender:'m'}, {$inc:{vampires:5}}, {multi:true});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn>
```

Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({gender:'m'}, {_id:0, name:1, vampires:1});
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Barney', vampires: 5 }
]
learn>
```

3.3.5

Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

```
learn> db.towns.update({name:'Portland'}, {$unset:{'mayor.party': ""}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

Проверить содержимое коллекции towns.

```
learn> db.towns.find({name:'Portland'});
[
  {
    _id: ObjectId('6a204cbd6179384a32abc122'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> _
```

3.3.6

Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.

```
]
learn> db.unicorns.update({name:'Pilot'}, {$push:{loves:'chocolate'}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

Проверить содержимое коллекции *unicorns*.

```
]
learn> db.unicorns.find({name:'Pilot'});
[
  {
    _id: ObjectId('6a2040e16179384a32abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> █
```

3.3.7

Изменить информацию о самке единорога *Aurora*: теперь она любит еще и сахар, и лимоны.

```
]
learn> db.unicorns.update({name:'Aurora'}, {$addToSet:{loves:{$each:['sugar','lemon']}}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

Проверить содержимое коллекции *unicorns*.

```
]
learn> db.unicorns.find({name:'Aurora'});
[
  {
    _id: ObjectId('6a2040b86179384a32abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> █
```

3.4.1

Создайте коллекцию towns, включающую следующие документы:

```
learn> db.towns.insert((name:"Punxsutawney", populatiuon: 6200, last_sensus: ISODate("2008-01-31"), famous_for:["phil the groundhog"], mayor:{name:"Jim Wehrle"}));
{ acknowledged: true, insertedIds: { '0': ObjectId('6a20552f6179384a32abc124') } }
learn> db.towns.insert((name:"New York", populatiuon: 22200000, last_sensus: ISODate("2009-07-31"), famous_for:["status of liberty","food"], mayor:{name:"Michael Bloomberg", party:"I"}));
{ acknowledged: true, insertedIds: { '0': ObjectId('6a2055466179384a32abc125') } }
learn> db.towns.insert((name:"Portland", populatiuon: 528000, last_sensus: ISODate("2009-07-20"), famous_for:["beer","food"], mayor:{name:"Sam Adams", party:"D"}));
{ acknowledged: true, insertedIds: { '0': ObjectId('6a20555c6179384a32abc126') } }
learn>
```

Удалите документы с беспартийными мэрами.

```
learn> db.towns.remove({"mayor.party":{"exists:false"}});
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 3 }
learn>
```

Проверьте содержание коллекции.

```
learn> db.towns.find();
[
  {
    _id: ObjectId('6a204ca56179384a32abc121'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a2055466179384a32abc125'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a20555c6179384a32abc126'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> ▀
```

Очистите коллекцию.

```
learn> db.towns.remove({});
{ acknowledged: true, deletedCount: 3 }
learn> ▀
```

Просмотрите список доступных коллекций.

```
learn> show collections
towns
unicorns
learn>
```

4.1.1

Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
learn> db.habitats.insert({_id:"forest", name:"Лес", desc:"Зона с деревьями"});
{ acknowledged: true, insertedIds: { '0': 'forest' } }
learn> db.habitats.insert({_id:"mountain", name:"Горы", desc:"Зона с высокими резкими склонами"});
{ acknowledged: true, insertedIds: { '0': 'mountain' } }
learn> db.habitats.insert({_id:"meadow", name:"Луг", desc:"Открытая равнина с травой"});
{ acknowledged: true, insertedIds: { '0': 'meadow' } }
learn>
```

Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

```
learn> db.unicorns.updateOne({name:'Aurora'}, {$set:{habitat:{$ref:"habitats", $id:"forest"}}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({name:'Unicrom'}, {$set:{habitat:{$ref:"habitats", $id:"mountain"}}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({name:'Rooooooodles'}, {$set:{habitat:{$ref:"habitats", $id:"meadow"}}});
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> _
```

Проверьте содержание коллекции единорогов.

```
learn> db.unicorns.find({name:'Aurora'}).pretty()
[
  {
    _id: ObjectId('6a2040b86179384a32abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    habitat: DBRef('habitats', 'forest')
  }
]
learn> _
```

4.2.1

Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
learn> db.unicorns.createIndex({name:1}, {unique:true})
name_1
learn> db.unicorns.insertOne({name:'Aurora', gender:'f'})
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Aurora" }
learn>
```

4.3.1

Получите информацию о всех индексах коллекции `unicorns`.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn>
```

Удалите все индексы, кроме индекса для идентификатора.

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn>
```

Попытайтесь удалить индекс для идентификатора.

```
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

4.4.1

Создайте объемную коллекцию `numbers`, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
learn> for(i=0; i<100000; i++){db.numbers.insert({value:i})}
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a205a0b6179384a32ad47c7') }
}
learn>
```

Выберите последних четыре документа.

```
learn> db.numbers.find().sort({$natural:-1}).limit(4)
[
  { _id: ObjectId('6a205a0b6179384a32ad47c7'), value: 99999 },
  { _id: ObjectId('6a205a0b6179384a32ad47c6'), value: 99998 },
  { _id: ObjectId('6a205a0b6179384a32ad47c5'), value: 99997 },
  { _id: ObjectId('6a205a0b6179384a32ad47c4'), value: 99996 }
]
learn>
```

Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)

`executionTimeMillis`: 53

```
learn> db.numbers.explain("executionStats").find().sort({value:-1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'BA27D965',
    planCacheShapeHash: 'BA27D965',
    planCacheKey: '7A892B81',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 53,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 42,
      works: 100006,
      advanced: 4,
      needTime: 100001,
      needYield: 0,
      saveState: 3,
      restoreState: 3,
      isEOF: 1,
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      totalDataSizeSorted: 260,
      usedDisk: false,
      spills: 0,
      spilledDataStorageSize: 0,
      inputStage: {
        stage: 'COLLSCAN',
        nReturned: 100000,
        executionTimeMillisEstimate: 0,
        works: 100001,
        advanced: 100000,
        needTime: 0,
        needYield: 0,
        saveState: 3,
        restoreState: 3,
        isEOF: 1,
        direction: 'forward',
        docsExamined: 100000
      }
    }
  }
}
```

```

    },
  },
  queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
  command: {
    find: 'numbers',
    filter: {},
    sort: { value: -1 },
    limit: 4,
    '$db': 'learn'
  },
  serverInfo: {
    host: 'DESKTOP-BI10QDL',
    port: 27017,
    version: '8.0.23',
    gitVersion: 'ccf0d81588377542f00eeecf1b1ffbc095b1eeff'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
learn>

```

Создайте индекс для ключа `value`.

```

learn> db.numbers.createIndex({value:1})
value_1
learn>

```

Получите информацию о всех индексах коллекции `numbers`.

```

learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn>

```

Выполните запрос 2.

```

learn> db.numbers.find().sort({value:-1}).limit(4)
[
  { _id: ObjectId('6a205a0b6179384a32ad47c7'), value: 99999 },
  { _id: ObjectId('6a205a0b6179384a32ad47c6'), value: 99998 },
  { _id: ObjectId('6a205a0b6179384a32ad47c5'), value: 99997 },
  { _id: ObjectId('6a205a0b6179384a32ad47c4'), value: 99996 }
]
learn>

```

Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

executionTimeMillis: 0

```

learn> db.numbers.explain("executionStats").find().sort({value:-1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'BA27D965',
    planCacheShapeHash: 'BA27D965',
    planCacheKey: '7A892B81',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nReturned: 4,
        executionTimeMillisEstimate: 0,
        works: 4,
        advanced: 4,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,

```

```

    saveState: 0,
    restoreState: 0,
    isEOF: 0,
    docsExamined: 4,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 4,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 0,
      keyPattern: { value: 1 },
      indexName: 'value_1',
      isMultiKey: false,
      multiKeyPaths: { value: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'backward',
      indexBounds: { value: [ '[MaxKey, MinKey]' ] },
      keysExamined: 4,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
},
queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
command: {
  find: 'numbers',
  filter: {},
  sort: { value: -1 },
  limit: 4,
  '$db': 'learn'
},
serverInfo: {
  host: 'DESKTOP-BI10QDL',
  port: 27017,
  version: '8.0.23',
  gitVersion: 'ccf0d81588377542f00eeecf1b1ffbc095b1eeff'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
learn>

```

Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Запрос с индексом значительно эффективнее. Время выполнения сократилось с 53 мс до 0 мс (фактически менее 1 мс), что означает ускорение более чем в 53 раза.

Вывод

1. В ходе лабораторной работы были освоены практические навыки установки и настройки СУБД MongoDB 8.0. Изучена архитектура документо-ориентированных баз данных, основанная на формате BSON.
2. Выполнены операции CRUD (Create, Read, Update, Delete): вставка документов, использование курсоров, выборка с фильтрацией (логические операторы \$and, \$or, \$in, \$all), обновление данных (операторы \$set, \$inc, \$push, \$addToSet) и удаление коллекций.
3. Изучены методы работы со сложной структурой данных: вложенные объекты, массивы и ссылки между коллекциями (DBRef). Освоена агрегация данных с помощью оператора \$group.
4. Экспериментально подтверждена эффективность использования индексов. При работе с коллекцией из 100 000 документов создание индекса по ключу сортировки позволило снизить время выполнения запроса с 53 мс до 0 мс и изменить тип сканирования с полного (COLLSCAN) на индексный (IXSCAN).
5. Выявлены особенности версии MongoDB 8.0: удаление устаревших методов (save, remove) и необходимость использования их современных аналогов (insertOne, deleteMany).